ECA5801-IOE AND AI DRIVEN SYSTEM

EXPERIMENT 6:

a) **Simulation and Performance Evaluation of People–Process–Data–Things IoE Framework using Edge/Fog-to-Cloud Pipeline with MQTT, Node-RED.**

PROGRAM CODE:

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt


np.random.seed(42)


# Simulation parameters

NUM_MESSAGES = 1000

MESSAGE_SIZE_BYTES = 1024

SIMULATION_TIME = 60


# P-P-D-T framework

print("PEOPLE-PROCESS-DATA-THINGS IoE FRAMEWORK")

print("=" * 50)

print("People: Users, Operators, Administrators")

print("Process: Monitoring, Analysis, Decision Making")

print("Data: Sensor readings and MQTT messages")

print("Things: IoE sensors and connected devices")


# Edge, Fog and Cloud parameters

architectures = {
```

```
    "Edge": {
        "network_delay": 5,
        "processing_time": 3,
        "packet_loss": 0.01
    },
    "Fog": {
        "network_delay": 15,
        "processing_time": 6,
        "packet_loss": 0.02
    },
    "Cloud": {
        "network_delay": 50,
        "processing_time": 12,
        "packet_loss": 0.03
    }
}

results = []

for architecture, params in architectures.items():

    network_delay = np.maximum(
        np.random.normal(
            params["network_delay"],
            params["network_delay"] * 0.15,
            NUM_MESSAGES
        ),
        0.1
    )
```

```
processing_time = np.maximum(
    np.random.normal(
        params["processing_time"],
        params["processing_time"] * 0.2,
        NUM_MESSAGES
    ),
    0.1
)

total_latency = network_delay + processing_time

delivered = np.random.random(NUM_MESSAGES) >= params["packet_loss"]

successful_messages = np.sum(delivered)
lost_messages = NUM_MESSAGES - successful_messages

average_latency = (
    np.mean(total_latency[delivered])
    if successful_messages > 0 else 0
)

throughput = (
    successful_messages * MESSAGE_SIZE_BYTES * 8
) / SIMULATION_TIME

reliability = (successful_messages / NUM_MESSAGES) * 100

average_processing = np.mean(processing_time)
```

```
    results.append({
        "Architecture": architecture,
        "Network Delay (ms)": np.mean(network_delay),
        "Processing Time (ms)": average_processing,
        "End-to-End Latency (ms)": average_latency,
        "Throughput (bps)": throughput,
        "Reliability (%)": reliability,
        "Messages Delivered": successful_messages,
        "Messages Lost": lost_messages
    })

df = pd.DataFrame(results)

print("\nSIMULATION RESULTS")
display(df.round(3))

df.to_csv("experiment_a_results.csv", index=False)

# Graph 1: Latency
plt.figure(figsize=(8, 5))
plt.bar(df["Architecture"], df["End-to-End Latency (ms)"])
plt.title("Edge, Fog and Cloud Latency Comparison")
plt.xlabel("Architecture")
plt.ylabel("Latency (ms)")
plt.grid(axis="y", alpha=0.3)
plt.show()

# Graph 2: Throughput
```

```
plt.figure(figsize=(8, 5))
plt.bar(df["Architecture"], df["Throughput (bps)"])
plt.title("Edge, Fog and Cloud Throughput Comparison")
plt.xlabel("Architecture")
plt.ylabel("Throughput (bps)")
plt.grid(axis="y", alpha=0.3)
plt.show()

# Graph 3: Reliability
plt.figure(figsize=(8, 5))
plt.bar(df["Architecture"], df["Reliability (%)"])
plt.title("Edge, Fog and Cloud Reliability Comparison")
plt.xlabel("Architecture")
plt.ylabel("Reliability (%)")
plt.ylim(0, 100)
plt.grid(axis="y", alpha=0.3)
plt.show()

# Graph 4: Processing Time
plt.figure(figsize=(8, 5))
plt.bar(df["Architecture"], df["Processing Time (ms)"])
plt.title("Edge, Fog and Cloud Processing Time")
plt.xlabel("Architecture")
plt.ylabel("Processing Time (ms)")
plt.grid(axis="y", alpha=0.3)
plt.show()

# P-P-D-T data flow
stages = ["People", "Process", "Data", "Things", "Edge/Fog", "Cloud"]
```

```
plt.figure(figsize=(12, 3))
positions = np.arange(len(stages))

plt.plot(positions, np.zeros(len(stages)), marker="o")

for i, stage in enumerate(stages):
    plt.text(i, 0.03, stage, ha="center")

for i in range(len(stages) - 1):
    plt.annotate(
        "",
        xy=(i + 0.9, 0),
        xytext=(i + 0.1, 0),
        arrowprops=dict(arrowstyle="->")
    )

plt.title("P-P-D-T IoE Data Flow")
plt.axis("off")
plt.show()

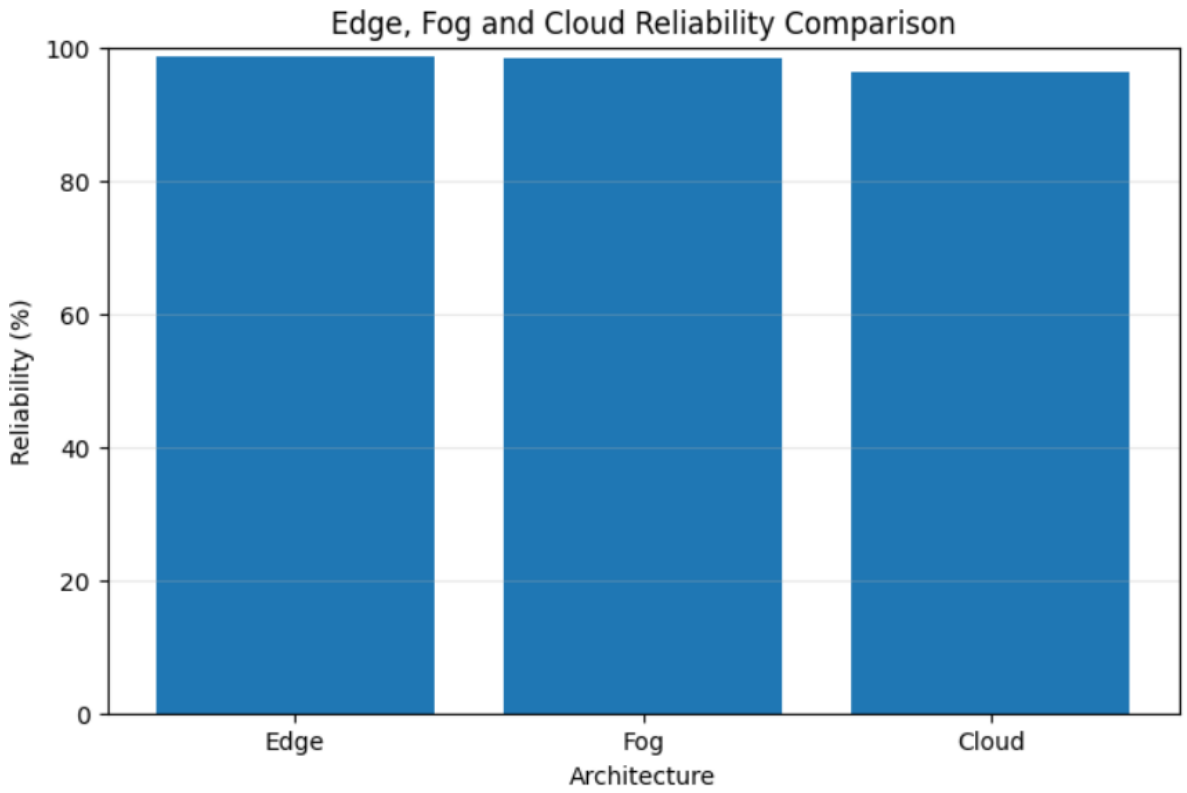
print("\nExperiment (a) completed successfully.")
print("Results saved to experiment_a_results.csv")
```

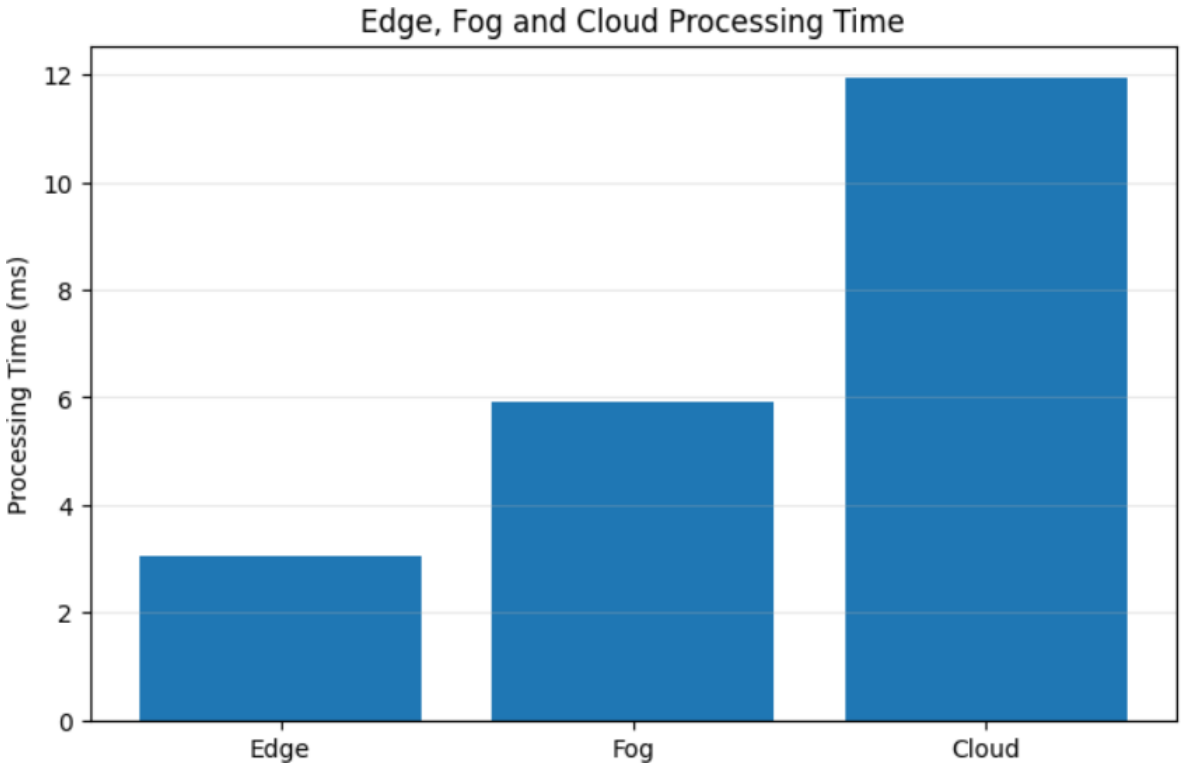
OUTPUT:

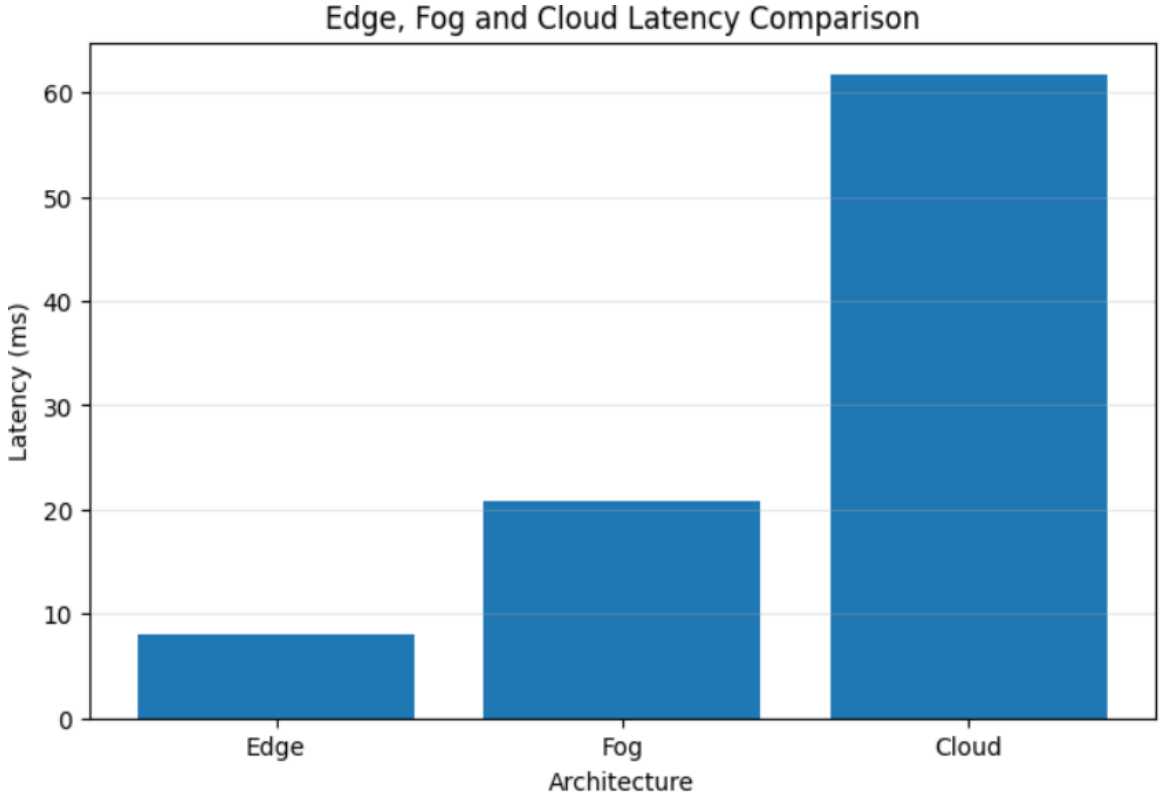
```
PEOPLE-PROCESS-DATA-THINGS IoE FRAMEWORK
==================================================
People: Users, Operators, Administrators
Process: Monitoring, Analysis, Decision Making
Data: Sensor readings and MQTT messages
Things: IoE sensors and connected devices

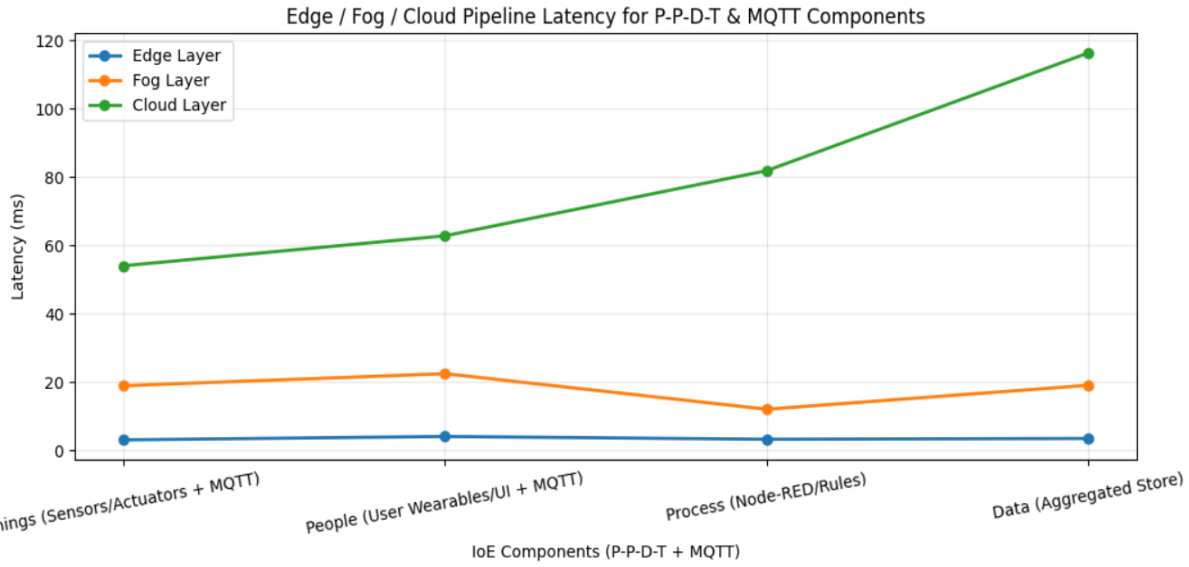
SIMULATION RESULTS
```

	Architecture	Network Delay (ms)	Processing Time (ms)	End-to-End Latency (ms)	Throughput (bps)	Reliability (%)	Messages Delivered	Messages Lost
0	Edge	5.014	3.043	8.055	134485.333	98.5	985	15
1	Fog	14.986	5.922	20.909	134212.267	98.3	983	17
2	Cloud	49.738	11.935	61.721	131618.133	96.4	964	36




Edge, Fog and Cloud Processing Time
Processing Time (ms)
0
2
4
6
8
10
12
Edge
Fog
Cloud


Edge, Fog and Cloud Latency Comparison
Latency (ms)
0
10
20
30
40
50
60
Edge
Fog
Cloud
Architecture



EXPERIMENT 6:

b) Benchmarking of 5G/6G Communication for IoE Applications based on Throughput, Latency, Reliability and Energy Efficiency

PROGRAM CODE:

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt


np.random.seed(42)


# Simulation parameters

NUM_MESSAGES = 2000

MESSAGE_SIZE_BYTES = 1500

SIMULATION_TIME = 60

NUM_TRIALS = 10


# Illustrative simulation profiles, not actual measured specifications

networks = {

    "5G": {
```

```
        "bandwidth_mbps": 100,
        "latency_ms": 20
"jitter_ms": 4,
        "packet_loss": 0.02,
        "power_watts": 5
    },
    "6G Hypothetical": {
        "bandwidth_mbps": 1000,
        "latency_ms": 5,
        "jitter_ms": 1,
        "packet_loss": 0.005,
        "power_watts": 8
    }
}

results = []

for network_name, params in networks.items():

    for trial in range(1, NUM_TRIALS + 1):

        latency = np.maximum(
            np.random.normal(
                params["latency_ms"],
                params["jitter_ms"],
                NUM_MESSAGES
            ),
            0.1
        )

        delivered = np.random.random(NUM_MESSAGES) >= params["packet_loss"]
```

```
successful_messages = np.sum(delivered)

delivered_bits = (
    successful_messages * MESSAGE_SIZE_BYTES * 8
)

throughput_mbps = (
    delivered_bits / SIMULATION_TIME
) / 1_000_000

average_latency = (
    np.mean(latency[delivered])
    if successful_messages > 0 else 0
)

reliability = (
    successful_messages / NUM_MESSAGES
) * 100

energy_joules = params["power_watts"] * SIMULATION_TIME

energy_efficiency = delivered_bits / energy_joules

results.append({
    "Network": network_name,
    "Trial": trial,
    "Latency (ms)": average_latency,
    "Throughput (Mbps)": throughput_mbps,
```

```
            "Reliability (%)": reliability,
            "Power (W)": params["power_watts"],
            "Energy (J)": energy_joules,
            "Energy Efficiency (bits/J)": energy_efficiency
        })

df = pd.DataFrame(results)

print("5G/6G SIMULATION RESULTS")
display(df.round(4))

df.to_csv("experiment_b_trial_results.csv", index=False)

# Calculate average performance across trials
summary = df.groupby("Network").agg({
    "Latency (ms)": "mean",
    "Throughput (Mbps)": "mean",
    "Reliability (%)": "mean",
    "Energy Efficiency (bits/J)": "mean"
}).reset_index()

print("\nAVERAGE PERFORMANCE COMPARISON")
display(summary.round(4))

summary.to_csv("experiment_b_summary.csv",  index=False)

# Graph 1: Throughput
plt.figure(figsize=(8, 5))
plt.bar(summary["Network"], summary["Throughput (Mbps)"])
```

```
plt.title("5G vs Hypothetical 6G Throughput")
plt.xlabel("Network")
plt.ylabel("Throughput (Mbps)")
plt.grid(axis="y", alpha=0.3)
plt.show()

# Graph 2: Latency
plt.figure(figsize=(8, 5))
plt.bar(summary["Network"], summary["Latency (ms)"])
plt.title("5G vs Hypothetical 6G Latency")
plt.xlabel("Network")
plt.ylabel("Latency (ms)")
plt.grid(axis="y", alpha=0.3)
plt.show()

# Graph 3: Reliability
plt.figure(figsize=(8, 5))
plt.bar(summary["Network"], summary["Reliability (%)"])
plt.title("5G vs Hypothetical 6G Reliability")
plt.xlabel("Network")
plt.ylabel("Reliability (%)")
plt.ylim(0, 100)
plt.grid(axis="y", alpha=0.3)
plt.show()

# Graph 4: Energy Efficiency
plt.figure(figsize=(8, 5))
plt.bar(
    summary["Network"],
```

```
    summary["Energy Efficiency (bits/J)"
    ]
)
plt.title("5G vs Hypothetical 6G Energy Efficiency")
plt.xlabel("Network")
plt.ylabel("Energy Efficiency (bits/J)")
plt.grid(axis="y", alpha=0.3)
plt.show()

print("\nExperiment (b) completed successfully.")
print("Results saved to experiment_b_trial_results.csv")
print("Summary saved to experiment_b_summary.csv")
```

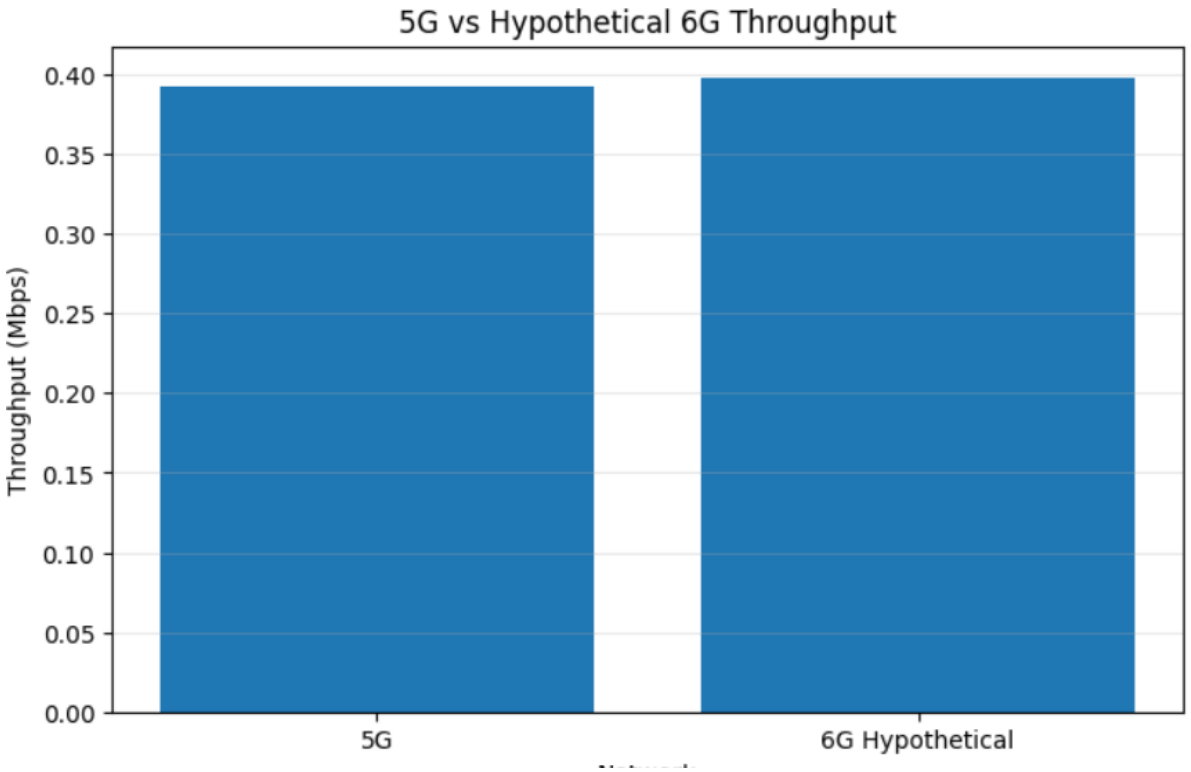
OUTPUT:

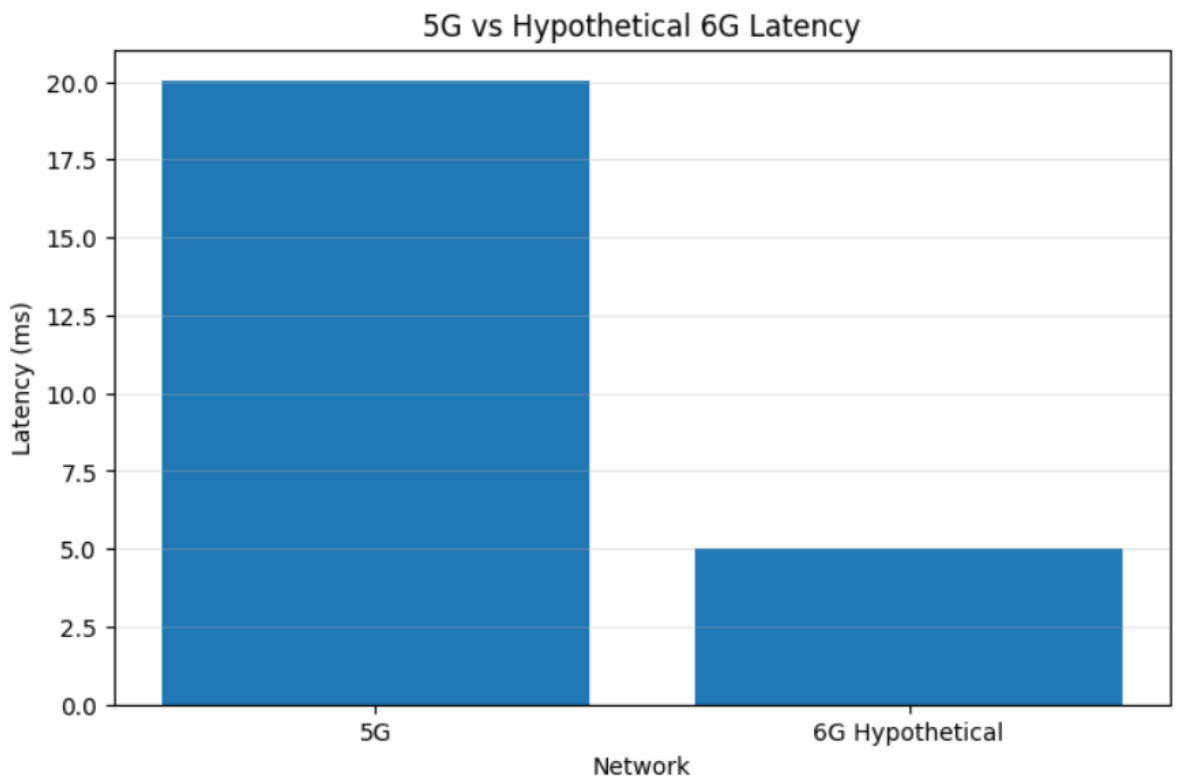
5G/6G SIMULATION RESULTS

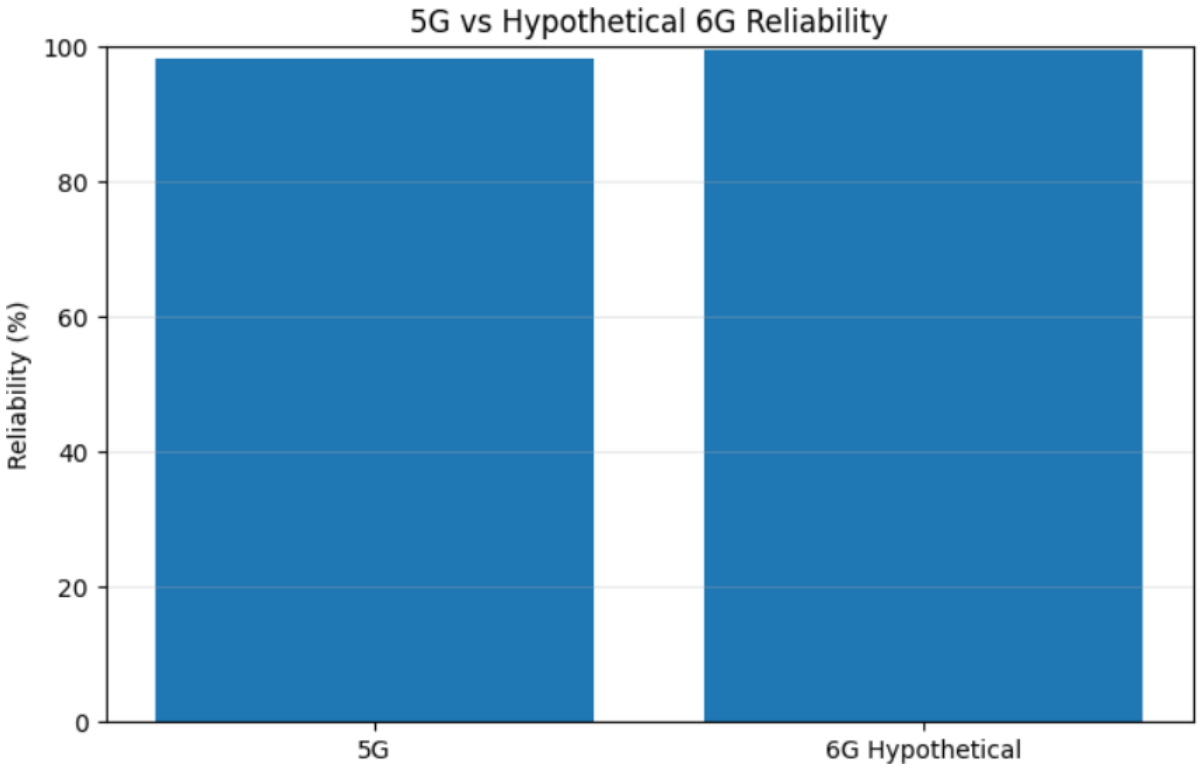
	Network	Trial	Latency (ms)	Throughput (Mbps)	Reliability (%)	Power (W)	Energy (J)	Energy Efficiency (bits/J)
0	5G	1	20.1843	0.3908	97.70	5	300	78160.0
1	5G	2	19.8477	0.3920	98.00	5	300	78400.0
2	5G	3	20.1075	0.3918	97.95	5	300	78360.0
3	5G	4	19.9550	0.3926	98.15	5	300	78520.0
4	5G	5	20.0507	0.3910	97.75	5	300	78200.0
5	5G	6	20.0635	0.3944	98.60	5	300	78880.0
6	5G	7	19.9432	0.3926	98.15	5	300	78520.0
7	5G	8	20.1182	0.3908	97.70	5	300	78160.0
8	5G	9	19.9775	0.3916	97.90	5	300	78320.0
9	5G	10	20.0487	0.3940	98.50	5	300	78800.0
10	6G Hypothetical	1	4.9908	0.3964	99.10	8	480	49550.0
11	6G Hypothetical	2	5.0345	0.3978	99.45	8	480	49725.0
12	6G Hypothetical	3	5.0190	0.3982	99.55	8	480	49775.0
13	6G Hypothetical	4	4.9586	0.3972	99.30	8	480	49650.0

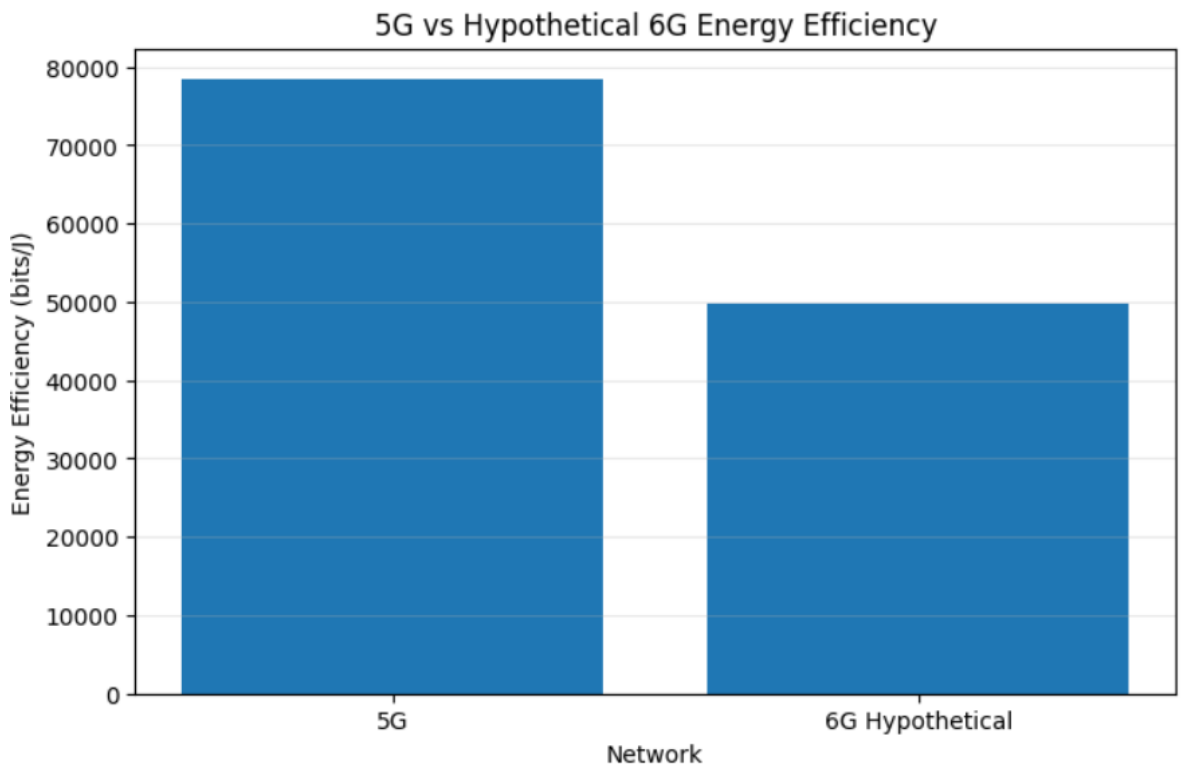
AVERAGE PERFORMANCE COMPARISON

	Network	Latency (ms)	Throughput (Mbps)	Reliability (%)	Energy Efficiency (bits/J)
0	5G	20.0296	0.3922	98.04	78432.0
1	6G Hypothetical	5.0014	0.3976	99.40	49700.0


5G vs Hypothetical 6G Throughput
Throughput (Mbps)
0.40
0.35
0.30
0.25
0.20
0.15
0.10
0.05
0.00
5G
6G Hypothetical


5G vs Hypothetical 6G Latency
Latency (ms)
20.0
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0.0
5G
6G Hypothetical
Network


5G vs Hypothetical 6G Reliability
Reliability (%)
100
80
60
40
20
0
5G
6G Hypothetical


5G vs Hypothetical 6G Energy Efficiency
Energy Efficiency (bits/J)
80000
70000
60000
50000
40000
30000
20000
10000
0
5G
6G Hypothetical
Network